Week 2 Assignment

NumPy and Pandas for Data Analytics

Name: Shyam Singh | College: RATM | Branch: Computer Science

This report contains the Python source code, program output, and a short explanation for each of the 10 tasks. Tasks 1 to 9 use small practice datasets. Task 10 is a mini data analysis project on an e-commerce dataset, and its key findings are given at the end of the report.

Task 1: NumPy Introduction & Arrays

Create a Python program using NumPy that:
Imports NumPy.
Creates a NumPy array containing at least 10 numbers.
Displays the array.
Displays the array's shape, size, and data type.
Creates a one-dimensional and two-dimensional array.

Source Code:

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

print("NumPy Array:")
print(arr)

print("Shape of Array:", arr.shape)
print("Size of Array:", arr.size)
print("Data Type of Array:", arr.dtype)

arr_1d = np.array([1, 2, 3, 4, 5])
print("\nOne-Dimensional Array:")
print(arr_1d)
arr_2d = np.array([[1, 2, 3],[4, 5, 6]])

print("\n Two-Dimensional Array:")
print(arr_2d)
```

Output:

```
NumPy Array:
[ 10  20  30  40  50  60  70  80  90 100]
Shape of Array: (10,)
Size of Array: 10
Data Type of Array: int64

One-Dimensional Array:
[1 2 3 4 5]

 Two-Dimensional Array:
[[1 2 3]
 [4 5 6]]
```

Explanation

First, I imported the NumPy library as np. Then, I created a NumPy array of 10 numbers (10 to 100) and printed it. After that, I displayed its shape, size, and data type using the shape, size, and dtype attributes. Finally, I created one one-dimensional array and one two-dimensional array and printed both of them. This task helped me understand how to create arrays in NumPy and check their basic properties.

Task 2: NumPy Indexing, Slicing & Reshaping

Using a NumPy array:
Access specific elements using indexing.
Extract a portion of the array using slicing.
Create a two-dimensional array.
Access specific rows and columns.
Reshape an array into different dimensions.
Display the original and reshaped arrays.

Source Code:

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

# 1. Display the original array
print("Original Array:")
print(arr)

# 2. Access specific elements using indexing
print("\nElement at index 0:", arr[0])
print("Element at index 4:", arr[4])
print("Last element:", arr[-1])

# 3. Extract a portion of the array using slicing
print("\n Sliced Array : ")
print(arr[2:7])

# 4. Create a two-dimensional array
arr_2d = np.array([[10, 20, 30],[40, 50, 60],[70, 80, 90]])

print("\n Two-Dimensional Array:")
print(arr_2d)

# 5. Access specific rows and columns
print("\n First Row:")
print(arr_2d[0])

print("\n Second Column:")
print(arr_2d[:, 1])

print("\nElement at Row 2, Column 3:")
print(arr_2d[1, 2])

# 6. Reshape the original array
reshaped_arr = arr.reshape(2, 5)

print("\nOriginal Array Before Reshaping:")
print(arr)

print("\nReshaped Array (2 Rows, 5 Columns):")
print(reshaped_arr)
# 7. Reshape into a different dimension
reshaped_arr_3d = arr.reshape(2, 1, 5)

print("\nReshaped Array (3 Dimensions):")
print(reshaped_arr_3d)
```

Output:

```
Original Array:
[ 10  20  30  40  50  60  70  80  90 100]

Element at index 0: 10
Element at index 4: 50
Last element: 100

 Sliced Array :
[30 40 50 60 70]

 Two-Dimensional Array:
[[10 20 30]
 [40 50 60]
 [70 80 90]]
 First Row:
[10 20 30]

 Second Column:
[20 50 80]

Element at Row 2, Column 3:
60

Original Array Before Reshaping:
[ 10  20  30  40  50  60  70  80  90 100]

Reshaped Array (2 Rows, 5 Columns):
[[ 10  20  30  40  50]
 [ 60  70  80  90 100]]

Reshaped Array (3 Dimensions):
[[[ 10  20  30  40  50]]

 [[ 60  70  80  90 100]]]
```

Explanation

First, I created a NumPy array of 10 numbers and displayed it. Then, I accessed elements using indexing, including the first element, the fifth element, and the last element using the negative index -1. After that, I used slicing arr[2:7] to extract a portion of the array. Next, I created a 3x3 two-dimensional array and accessed its first row, its second column, and a single element using row and column positions. Finally, I used reshape() to change the original array into a 2x5 array and then into a 3-dimensional (2, 1, 5) array, and printed the original and reshaped arrays.

Task 3: NumPy Mathematical & Statistical Operations

Perform the following operations using NumPy:
Mathematical Operations: Addition, Subtraction, Multiplication, Division
Statistical Functions: Mean, Median, Minimum, Maximum, Standard Deviation, Sum
Use a numerical dataset of your choice and display the results.

Source Code:

```
import numpy as np

# Create two numerical arrays
arr1 = np.array([10, 20, 30, 40, 50])
arr2 = np.array([2, 4, 6, 8, 10])

# Display the arrays
print("Array 1:", arr1)
print("Array 2:", arr2)

# Mathematical Operations
print("\n Mathematical Operations ")

# Addition
print("Addition:", np.add(arr1, arr2))

# Subtraction
print("Subtraction:", np.subtract(arr1, arr2))

# Multiplication
print("Multiplication:", np.multiply(arr1, arr2))

# Division
print("Division:", np.divide(arr1, arr2))
# Statistical Operations
print("\n Statistical Operations ")
# Create a dataset for statistical calculations
data = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
print("Dataset:", data)
# Mean
print("Mean:", np.mean(data))
# Median
print("Median:", np.median(data))
# Minimum
print("Minimum:", np.min(data))
# Maximum
print("Maximum:", np.max(data))
# Standard Deviation
print("Standard Deviation:", np.std(data))
# Sum
print("Sum:", np.sum(data))
```

Output:

```
Array 1: [10 20 30 40 50]
Array 2: [ 2  4  6  8 10]

 Mathematical Operations
Addition: [12 24 36 48 60]
Subtraction: [ 8 16 24 32 40]
Multiplication: [ 20  80 180 320 500]
Division: [5. 5. 5. 5. 5.]

 Statistical Operations
Dataset: [ 10  20  30  40  50  60  70  80  90 100]
Mean: 55.0
Median: 55.0
Minimum: 10
```

```
Maximum: 100
Standard Deviation: 28.722813232690143
Sum: 550
```

Explanation

First, I created two NumPy arrays and performed addition, subtraction, multiplication, and division on them using np.add(), np.subtract(), np.multiply(), and np.divide(). These operations work element by element. Then, I created a dataset of 10 numbers and calculated its mean, median, minimum, maximum, standard deviation, and sum using the NumPy statistical functions. Finally, I displayed all the results using the print() function.

Task 4: Pandas Series & DataFrame

Create a Pandas program that:
Creates a Pandas Series.
Creates a DataFrame containing student or employee information.
Displays the DataFrame.
Displays the column names.
Displays the index.
Adds a new column to the DataFrame.
Displays the updated DataFrame.

Source Code:

```
import pandas as pd

# 1. Create a Pandas Series
marks = pd.Series([85, 90, 78, 92, 88])

print("Pandas Series:")
print(marks)

# 2. Create a DataFrame containing student information
data = {
    'Student_ID': [101, 102, 103, 104, 105],
    'Name': ['Rahul', 'Amit', 'Priya', 'Neha', 'Rohit'],
    'Age': [20, 21, 19, 22, 20],
    'Course': ['MCA', 'BCA', 'MCA', 'BBA', 'BCA']
}

df = pd.DataFrame(data)

# 3. Display the DataFrame
print("\nOriginal DataFrame:")
print(df)

# 4. Display the column names
print("\nColumn Names:")
print(df.columns)

# 5. Display the index
print("\nDataFrame Index:")
print(df.index)

# 6. Add a new column to the DataFrame
df['Marks'] = [85, 90, 78, 92, 88]

# 7. Display the updated DataFrame
print("\nUpdated DataFrame:")
print(df)
```

Output:

```
Pandas Series:
0    85
1    90
2    78
3    92
4    88
dtype: int64

Original DataFrame:
   Student_ID   Name  Age Course
0         101  Rahul   20    MCA
1         102   Amit   21    BCA
2         103  Priya   19    MCA
3         104   Neha   22    BBA
```

```
4         105  Rohit   20    BCA

Column Names:
Index(['Student_ID', 'Name', 'Age', 'Course'], dtype='object')

DataFrame Index:
RangeIndex(start=0, stop=5, step=1)

Updated DataFrame:
   Student_ID   Name  Age Course  Marks
0         101  Rahul   20    MCA     85
1         102   Amit   21    BCA     90
2         103  Priya   19    MCA     78
3         104   Neha   22    BBA     92
4         105  Rohit   20    BCA     88
```

Explanation

First, I imported Pandas and created a Series of marks. Then, I created a DataFrame of student information using a dictionary with Student_ID, Name, Age, and Course, and displayed it. After that, I displayed the column names using df.columns and the index using df.index. Finally, I added a new column named Marks to the DataFrame and displayed the updated DataFrame. This task helped me understand the difference between a Series and a DataFrame.

Task 5: Reading & Inspecting Data

Use a CSV dataset and perform the following operations:
Read the CSV file using Pandas.
Display the first 5 rows using head().
Display the last 5 rows using tail().
Check the number of rows and columns.
Display column names.
Check data types using dtypes.
Use info() and describe() to understand the dataset.

Source Code:

```
# Import Pandas
import pandas as pd

# Read the CSV file
df = pd.read_csv("student.csv")

# Display the first 5 rows
print("First 5 Rows:")
print(df.head())

# Display the last 5 rows
print("\nLast 5 Rows:")
print(df.tail())
# Check the number of rows and columns
print("\nNumber of Rows and Columns:")
print(df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns)

# Check data types
print("\nData Types:")
print(df.dtypes)

# Display dataset information
print("\nDataset Information:")
df.info()

# Display statistical summary
print("\nStatistical Summary:")
print(df.describe())
```

Output:

```
First 5 Rows:
   Student_ID   Name  Age  Marks  Attendance
0         101  Rahul   20     85          90
1         102   Amit   21     78          85
2         103  Priya   19     92          95
3         104   Neha   22     88          92
4         105  Rohit   20     75          80

Last 5 Rows:
   Student_ID    Name  Age  Marks  Attendance
5         106  Anjali   21     95          98
6         107   Vikas   23     69          75
7         108   Pooja   20     82          88
8         109   Arjun   22     90          93
```

```
9         110  Simran   19     87          91

Number of Rows and Columns:
(10, 5)

Column Names:
Index(['Student_ID', 'Name', 'Age', 'Marks', 'Attendance'], dtype='object')

Data Types:
Student_ID     int64
Name          object
Age            int64
Marks          int64
Attendance     int64
dtype: object

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Student_ID  10 non-null     int64
 1   Name        10 non-null     object
 2   Age         10 non-null     int64
 3   Marks       10 non-null     int64
 4   Attendance  10 non-null     int64
dtypes: int64(4), object(1)
memory usage: 532.0+ bytes

Statistical Summary:
       Student_ID        Age      Marks  Attendance
count    10.00000  10.000000  10.000000   10.000000
mean    105.50000  20.700000  84.100000   88.700000
std       3.02765   1.337494   8.116513    6.992853
min     101.00000  19.000000  69.000000   75.000000
25%     103.25000  20.000000  79.000000   85.750000
50%     105.50000  20.500000  86.000000   90.500000
75%     107.75000  21.750000  89.500000   92.750000
max     110.00000  23.000000  95.000000   98.000000
```

Explanation

First, I read the student.csv file using pd.read_csv(). Then, I displayed the first 5 rows using head() and the last 5 rows using tail(). After that, I checked the number of rows and columns using shape, which showed (10, 5), and displayed the column names. I also checked the data type of each column using dtypes. Finally, I used info() to see the non-null counts and memory usage, and describe() to see statistics such as mean, standard deviation, minimum, and maximum of the numeric columns.

Task 6: Selecting, Filtering & Sorting Data

Using a Pandas DataFrame:
Select specific columns.
Select specific rows.
Filter records based on a condition.
Apply multiple filtering conditions.
Sort data in ascending order.
Sort data in descending order.

Source Code:

```
import pandas as pd

# Read the CSV file
df = pd.read_csv("student.csv")

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# 1. Select specific columns
print("\n1. Selected Columns (Name and Marks):")
print(df[["Name", "Marks"]])

# 2. Select specific rows
print("\n2. Selected Rows (First 3 Students):")
print(df.iloc[0:3])

# 3. Filter records based on a condition
print("\n3. Students Scoring More Than 80 Marks:")
print(df[df["Marks"] > 80])

# 4. Apply multiple filtering conditions
print("\n4. Students with Marks > 80 and Attendance >= 90:")
print(df[(df["Marks"] > 80) & (df["Attendance"] >= 90)])

# 5. Sort data in ascending order
print("\n5. Students Sorted by Marks (Ascending):")
print(df.sort_values(by="Marks", ascending=True))
# 6. Sort data in descending order
print("\n6. Students Sorted by Marks (Descending):")
print(df.sort_values(by="Marks", ascending=False))
```

Output:

```
Original DataFrame:
   Student_ID    Name  Age  Marks  Attendance
0         101   Rahul   20     85          90
1         102    Amit   21     78          85
2         103   Priya   19     92          95
3         104    Neha   22     88          92
4         105   Rohit   20     75          80
5         106  Anjali   21     95          98
6         107   Vikas   23     69          75
7         108   Pooja   20     82          88
8         109   Arjun   22     90          93
9         110  Simran   19     87          91

1. Selected Columns (Name and Marks):
```

```
     Name  Marks
0   Rahul     85
1    Amit     78
2   Priya     92
3    Neha     88
4   Rohit     75
5  Anjali     95
6   Vikas     69
7   Pooja     82
8   Arjun     90
9  Simran     87

2. Selected Rows (First 3 Students):
   Student_ID   Name  Age  Marks  Attendance
0         101  Rahul   20     85          90
1         102   Amit   21     78          85
2         103  Priya   19     92          95

3. Students Scoring More Than 80 Marks:
   Student_ID    Name  Age  Marks  Attendance
0         101   Rahul   20     85          90
2         103   Priya   19     92          95
3         104    Neha   22     88          92
5         106  Anjali   21     95          98
7         108   Pooja   20     82          88
8         109   Arjun   22     90          93
9         110  Simran   19     87          91
4. Students with Marks > 80 and Attendance >= 90:
   Student_ID    Name  Age  Marks  Attendance
0         101   Rahul   20     85          90
2         103   Priya   19     92          95
3         104    Neha   22     88          92
5         106  Anjali   21     95          98
8         109   Arjun   22     90          93
9         110  Simran   19     87          91

5. Students Sorted by Marks (Ascending):
   Student_ID    Name  Age  Marks  Attendance
6         107   Vikas   23     69          75
4         105   Rohit   20     75          80
1         102    Amit   21     78          85
7         108   Pooja   20     82          88
0         101   Rahul   20     85          90
9         110  Simran   19     87          91
3         104    Neha   22     88          92
8         109   Arjun   22     90          93
2         103   Priya   19     92          95
5         106  Anjali   21     95          98

6. Students Sorted by Marks (Descending):
   Student_ID    Name  Age  Marks  Attendance
5         106  Anjali   21     95          98
2         103   Priya   19     92          95
8         109   Arjun   22     90          93
3         104    Neha   22     88          92
9         110  Simran   19     87          91
0         101   Rahul   20     85          90
7         108   Pooja   20     82          88
1         102    Amit   21     78          85
4         105   Rohit   20     75          80
6         107   Vikas   23     69          75
```

Explanation

First, I read the student dataset into a DataFrame. Then, I selected the Name and Marks columns using df[['Name', 'Marks']] and the first three rows using iloc[0:3]. After that, I filtered the students who scored more than 80 marks, and then applied two conditions together using the & operator (Marks > 80 and Attendance >= 90). Finally, I used sort_values() to sort the students by Marks in ascending and then in descending order.

Task 7: Handling Missing Values

Take a dataset containing missing values and perform the following:
Identify missing values using isnull() / isna().
Count missing values in each column.
Remove rows containing missing values.
Fill missing values using appropriate methods.
Display the dataset before and after handling missing values.
Briefly explain why handling missing data is important in Data Analytics.

Source Code:

```
import pandas as pd
import numpy as np

# Create a dataset containing missing values
data = {
    'Student_ID': [101, 102, 103, 104, 105, 106],
    'Name': ['Rahul', 'Amit', 'Priya', 'Neha', 'Rohit', 'Anjali'],
    'Age': [20, 21, np.nan, 22, 20, np.nan],
    'Marks': [85, np.nan, 92, 88, np.nan, 95],
    'Attendance': [90, 85, 95, np.nan, 80, 98]
}

df = pd.DataFrame(data)
# 1. Display the original dataset
print("Original Dataset:")
print(df)

# 2. Identify missing values using isnull()
print("\nMissing Values:")
print(df.isnull())

# 3. Count missing values in each column
print("\nMissing Values Count:")
print(df.isnull().sum())

# 4. Remove rows containing missing values
df_removed = df.dropna()
print("\nDataset After Removing Missing Values:")
print(df_removed)

# 5. Fill missing values using appropriate methods
df_filled = df.copy()

# Fill missing Age values with the mean age
df_filled['Age'] = df_filled['Age'].fillna(df_filled['Age'].mean())

# Fill missing Marks values with the mean marks
df_filled['Marks'] = df_filled['Marks'].fillna(df_filled['Marks'].mean())

# Fill missing Attendance values with the mean attendance
df_filled['Attendance'] = df_filled['Attendance'].fillna(
    df_filled['Attendance'].mean()
)

# 6. Display the dataset after filling missing values
print("\nDataset After Filling Missing Values:")
print(df_filled)

# 7. Verify that no missing values remain
print("\nMissing Values After Filling:")
print(df_filled.isnull().sum())
```

Output:

```
Original Dataset:
   Student_ID    Name   Age  Marks  Attendance
0         101   Rahul  20.0   85.0        90.0
1         102    Amit  21.0    NaN        85.0
2         103   Priya   NaN   92.0        95.0
3         104    Neha  22.0   88.0         NaN
4         105   Rohit  20.0    NaN        80.0
5         106  Anjali   NaN   95.0        98.0

Missing Values:
   Student_ID   Name    Age  Marks  Attendance
0       False  False  False  False       False
1       False  False  False   True       False
2       False  False   True  False       False
3       False  False  False  False        True
4       False  False  False   True       False
5       False  False   True  False       False

Missing Values Count:
Student_ID    0
Name          0
Age           2
Marks         2
Attendance    1
dtype: int64

Dataset After Removing Missing Values:
   Student_ID   Name   Age  Marks  Attendance
0         101  Rahul  20.0   85.0        90.0

Dataset After Filling Missing Values:
   Student_ID    Name    Age  Marks  Attendance
0         101   Rahul  20.00   85.0        90.0
1         102    Amit  21.00   90.0        85.0
2         103   Priya  20.75   92.0        95.0
3         104    Neha  22.00   88.0        89.6
4         105   Rohit  20.00   90.0        80.0
5         106  Anjali  20.75   95.0        98.0

Missing Values After Filling:
Student_ID    0
Name          0
Age           0
Marks         0
Attendance    0
dtype: int64
```

Explanation

First, I created a student dataset that contains missing values (NaN) in the Age, Marks, and Attendance columns. Then, I used isnull() to find the missing values and isnull().sum() to count them in each column. After that, I used dropna() to remove rows with missing values; this left only one row out of six, which shows that removing rows can cause a big loss of data. So I also filled the missing values with the mean of each column using fillna(), and finally checked that no missing values remained.

Why handling missing data is important in Data Analytics:

Missing values can give wrong averages, totals, and charts, and many Pandas and machine learning functions do not work properly with NaN values. If we ignore them, the analysis can become biased and the conclusions can be wrong. Removing the rows is simple, but we may lose a lot of useful data (here 5 of 6 rows were lost). Filling the values with the mean or median keeps the dataset size, so the analysis stays more reliable.

Task 8: Merge, Concatenate, GroupBy & Pivot Table

Using two or more datasets, perform the following operations:
Merge two DataFrames using a common column.
Concatenate two DataFrames.
GroupBy: group data based on a selected column and calculate sum, mean, count, minimum, or maximum.
Pivot Table: create a Pivot Table using Pandas and summarize the data.
Display the output of each operation.

Source Code:

```
import pandas as pd

# Create the first DataFrame: Student Information
df1 = pd.DataFrame({
    'Student_ID': [101, 102, 103, 104, 105],
    'Name': ['Rahul', 'Amit', 'Priya', 'Neha', 'Rohit'],
    'Department': ['CS', 'IT', 'CS', 'IT', 'CS']
})

# Create the second DataFrame: Student Marks
df2 = pd.DataFrame({
    'Student_ID': [101, 102, 103, 104, 105],
    'Marks': [85, 78, 92, 88, 75],
    'Attendance': [90, 85, 95, 92, 80]
})

# Display original DataFrames
print("First DataFrame:")
print(df1)

print("\nSecond DataFrame:")
print(df2)


# 1. MERGE TWO DATAFRAMES
merged_df = pd.merge(df1, df2, on='Student_ID')

print("\n1. Merged DataFrame:")
print(merged_df)


# 2. CONCATENATE TWO DATAFRAMES
df3 = pd.DataFrame({
    'Student_ID': [106, 107],
    'Name': ['Anjali', 'Vikas'],
    'Department': ['IT', 'CS']
})

concatenated_df = pd.concat([df1, df3], ignore_index=True)

print("\n2. Concatenated DataFrame:")
print(concatenated_df)
# 3. GROUPBY DEPARTMENT
grouped_df = merged_df.groupby('Department')

print("\n3. GroupBy Department - Total Marks:")
print(grouped_df['Marks'].sum())


# 4. AGGREGATE FUNCTIONS
print("\n4. Aggregate Statistics by Department:")
print(
    merged_df.groupby('Department')['Marks'].agg(
        ['sum', 'mean', 'count', 'min', 'max']
```

```
    )
)


# 5. PIVOT TABLE
pivot_df = pd.pivot_table(
    merged_df,
    values='Marks',
    index='Department',
    columns='Name',
    aggfunc='mean',
    fill_value=0
)
print("\n5. Pivot Table:")
print(pivot_df)


# 6. PIVOT TABLE WITH MULTIPLE AGGREGATIONS
pivot_summary = pd.pivot_table(
    merged_df,
    values='Marks',
    index='Department',
    aggfunc=['sum', 'mean', 'count']
)

print("\n6. Pivot Table Summary:")
print(pivot_summary)
```

Output:

```
First DataFrame:
   Student_ID   Name Department
0         101  Rahul         CS
1         102   Amit         IT
2         103  Priya         CS
3         104   Neha         IT
4         105  Rohit         CS

Second DataFrame:
   Student_ID  Marks  Attendance
0         101     85          90
1         102     78          85
2         103     92          95
3         104     88          92
4         105     75          80

1. Merged DataFrame:
   Student_ID   Name Department  Marks  Attendance
0         101  Rahul         CS     85          90
1         102   Amit         IT     78          85
2         103  Priya         CS     92          95
3         104   Neha         IT     88          92
4         105  Rohit         CS     75          80

2. Concatenated DataFrame:
   Student_ID    Name Department
0         101   Rahul         CS
1         102    Amit         IT
```

```
2         103   Priya         CS
3         104    Neha         IT
4         105   Rohit         CS
5         106  Anjali         IT
6         107   Vikas         CS

3. GroupBy Department - Total Marks:
Department
CS    252
IT    166
Name: Marks, dtype: int64

4. Aggregate Statistics by Department:
            sum  mean  count  min  max
Department
CS          252  84.0      3   75   92
IT          166  83.0      2   78   88

5. Pivot Table:
Name        Amit  Neha  Priya  Rahul  Rohit
Department
CS           0.0   0.0   92.0   85.0   75.0
IT          78.0  88.0    0.0    0.0    0.0

6. Pivot Table Summary:
             sum  mean count
           Marks Marks Marks
Department
CS           252  84.0     3
IT           166  83.0     2
```

Explanation

First, I created two DataFrames: one with student information (Student_ID, Name, Department) and one with marks and attendance. Then, I merged them using the common column Student_ID with pd.merge(). After that, I created a third DataFrame with two new students and joined it with the first one using pd.concat(). Next, I grouped the merged data by Department using groupby() and calculated the sum, mean, count, minimum, and maximum of Marks. Finally, I created two pivot tables using pd.pivot_table(): one showing the marks of each student by department, and one showing the sum, mean, and count of marks by department.

Task 9: Exporting Data

After processing the dataset:
Export the final DataFrame to a new CSV file.
Verify that the exported file contains the processed data.
Save the file properly for submission.

Source Code:

```
import pandas as pd

# Create a DataFrame
data = {
    'Student_ID': [101, 102, 103, 104, 105],
    'Name': ['Rahul', 'Amit', 'Priya', 'Neha', 'Rohit'],
    'Age': [20, 21, 19, 22, 20],
    'Marks': [85, 78, 92, 88, 75],
    'Attendance': [90, 85, 95, 92, 80]
}

df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Process the DataFrame by sorting marks in descending order
df = df.sort_values(by='Marks', ascending=False)

# Reset the index
df = df.reset_index(drop=True)

# Display the processed DataFrame
print("\nProcessed DataFrame:")
print(df)

# Export the DataFrame to a CSV file
df.to_csv("processed_students.csv", index=False)

print("\nData exported successfully to processed_students.csv")

# Verify the exported file by reading it
df_exported = pd.read_csv("processed_students.csv")

# Display the exported data
print("\nVerified Exported Data:")
print(df_exported)

# Verify the shape of the exported DataFrame
print("\nExported Data Shape:", df_exported.shape)
# Check whether the exported data matches the processed data
if df.equals(df_exported):
    print("Verification Successful: Exported data matches the processed data.")
else:
    print("Verification Failed: Data does not match.")
```

Output:

```
Original DataFrame:
   Student_ID   Name  Age  Marks  Attendance
0         101  Rahul   20     85          90
1         102   Amit   21     78          85
2         103  Priya   19     92          95
3         104   Neha   22     88          92
4         105  Rohit   20     75          80

Processed DataFrame:
   Student_ID   Name  Age  Marks  Attendance
0         103  Priya   19     92          95
1         104   Neha   22     88          92
2         101  Rahul   20     85          90
3         102   Amit   21     78          85
4         105  Rohit   20     75          80

Data exported successfully to processed_students.csv

Verified Exported Data:
   Student_ID   Name  Age  Marks  Attendance
0         103  Priya   19     92          95
1         104   Neha   22     88          92
2         101  Rahul   20     85          90
3         102   Amit   21     78          85
4         105  Rohit   20     75          80

Exported Data Shape: (5, 5)
Verification Successful: Exported data matches the processed data.
```

Explanation

First, I created a student DataFrame and processed it by sorting the Marks column in descending order and resetting the index. Then, I exported the processed DataFrame to a new file named processed_students.csv using to_csv() with index=False, so the index column is not saved. After that, I read the exported file again using read_csv() and displayed it. Finally, I checked its shape and used df.equals() to confirm that the exported data matches the processed data, and the verification was successful.

Task 10: Mini Data Analysis Project (E-commerce Dataset)

Perform a basic data analysis project using NumPy and Pandas. Dataset used: E-commerce Dataset (ecommerce.csv, 1000 orders, 16 columns). The project includes:
Loading the dataset. Data inspection. Identifying and handling missing values.
Selecting and filtering data. Sorting data. Performing GroupBy analysis.
Creating at least one Pivot Table. Finding useful insights from the dataset.
Exporting the cleaned/processed dataset.

Source Code:

```
# ============================================================
# TASK 10: MINI E-COMMERCE DATA ANALYSIS PROJECT
# Libraries: NumPy, Pandas, Matplotlib, Seaborn
# Dataset: ecommerce.csv
# ============================================================

# 1. Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 2. Load the Dataset
df = pd.read_csv("ecommerce.csv")
print("=" * 60)
print("1. ORIGINAL DATASET")
print("=" * 60)

print(df.head())
print("\nDataset Shape:", df.shape)
print("\nColumn Names:")
print(df.columns.tolist())


# 3. Data Inspection
print("\n" + "=" * 60)
print("2. DATA INSPECTION")
print("=" * 60)

print("\nDataset Information:")
df.info()

print("\nStatistical Summary:")
print(df.describe())

print("\nData Types:")
print(df.dtypes)

print("\nDuplicate Rows:", df.duplicated().sum())
# 4. Identify Missing Values
print("\n" + "=" * 60)
print("3. MISSING VALUES")
print("=" * 60)

print(df.isnull().sum())

print("\nTotal Missing Values:", df.isnull().sum().sum())


# 5. Handle Missing Values
# Delivery_Date may be blank for undelivered or cancelled orders.
# Preserve these blanks because they have business meaning.
```

```
df["Order_Date"] = pd.to_datetime(df["Order_Date"], errors="coerce")
df["Delivery_Date"] = pd.to_datetime(df["Delivery_Date"], errors="coerce")

# Remove duplicate records if present
df = df.drop_duplicates()

# Fill missing categorical values if any exist
for column in ["Product", "Category", "Payment_Method",
               "Order_Status", "City", "State", "Sales_Channel"]:
    df[column] = df[column].fillna("Unknown")

# Fill missing numerical values using the median
numeric_columns = [
    "Quantity",
    "Unit_Price_INR",
    "Discount_Percent",
    "Discount_Amount_INR",
    "Total_Amount_INR"
]

for column in numeric_columns:
    df[column] = df[column].fillna(df[column].median())

print("\n" + "=" * 60)
print("4. CLEANED DATASET")
print("=" * 60)

print(df.head())
print("\nRemaining Missing Values:")
print(df.isnull().sum())


# 6. NumPy Mathematical and Statistical Analysis
print("\n" + "=" * 60)
print("5. NUMPY STATISTICAL ANALYSIS")
print("=" * 60)

sales = df["Total_Amount_INR"].to_numpy()
quantity = df["Quantity"].to_numpy()

print("Total Sales (INR):", round(np.sum(sales), 2))
print("Average Order Value (INR):", round(np.mean(sales), 2))
print("Median Order Value (INR):", round(np.median(sales), 2))
print("Minimum Order Value (INR):", round(np.min(sales), 2))
print("Maximum Order Value (INR):", round(np.max(sales), 2))
print("Sales Standard Deviation:", round(np.std(sales), 2))
print("Total Quantity Sold:", int(np.sum(quantity)))


# 7. Business KPIs
print("\n" + "=" * 60)
print("6. BUSINESS KPIs")
print("=" * 60)

total_orders = df["Order_ID"].nunique()
total_customers = df["Customer_ID"].nunique()
```

```python
total_products = df["Product"].nunique()

delivered_orders = (df["Order_Status"] == "Delivered").sum()
cancelled_orders = (df["Order_Status"] == "Cancelled").sum()
returned_orders = (df["Order_Status"] == "Returned").sum()

print("Total Orders:", total_orders)
print("Unique Customers:", total_customers)
print("Unique Products:", total_products)
print("Delivered Orders:", delivered_orders)
print("Cancelled Orders:", cancelled_orders)
print("Returned Orders:", returned_orders)

print(
    "Cancellation Rate (%):",
    round(cancelled_orders / total_orders * 100, 2)
)

print(
    "Return Rate (%):",
    round(returned_orders / total_orders * 100, 2)
)


# 8. Select Specific Columns
print("\n" + "=" * 60)
print("7. SELECTED COLUMNS")
print("=" * 60)
print(
    df[[
        "Order_ID",
        "Product",
        "Category",
        "Quantity",
        "Total_Amount_INR"
    ]].head(10)
)


# 9. Filter Records
print("\n" + "=" * 60)
print("8. FILTERING DATA")
print("=" * 60)

# Orders with value greater than INR 2000
high_value_orders = df[df["Total_Amount_INR"] > 2000]

print("\nOrders Above INR 2000:")
print(high_value_orders.head(10))

# Electronics orders
electronics_orders = df[df["Category"] == "Electronics"]

print("\nElectronics Orders:")
print(electronics_orders.head(10))
# Multiple conditions
high_value_delivered = df[
    (df["Total_Amount_INR"] > 2000) &
    (df["Order_Status"] == "Delivered")
]

print("\nHigh-Value Delivered Orders:")
print(high_value_delivered.head(10))


# 10. Sort Data
print("\n" + "=" * 60)
print("9. SORTING DATA")
```

```
print("=" * 60)

print("\nTop 10 Orders by Value:")
print(
    df.sort_values(
        by="Total_Amount_INR",
        ascending=False
    )[["Order_ID", "Product", "Total_Amount_INR"]].head(10)
)

print("\nLowest 10 Orders by Value:")
print(
    df.sort_values(
        by="Total_Amount_INR",
        ascending=True
    )[["Order_ID", "Product", "Total_Amount_INR"]].head(10)
)


# 11. GroupBy Analysis
print("\n" + "=" * 60)
print("10. CATEGORY-WISE ANALYSIS")
print("=" * 60)

category_analysis = df.groupby("Category").agg(
    Total_Orders=("Order_ID", "nunique"),
    Total_Quantity=("Quantity", "sum"),
    Total_Sales=("Total_Amount_INR", "sum"),
    Average_Order_Value=("Total_Amount_INR", "mean")
).sort_values(by="Total_Sales", ascending=False)

print(category_analysis.round(2))


# 12. Monthly Sales Analysis
print("\n" + "=" * 60)
print("11. MONTHLY SALES ANALYSIS")
print("=" * 60)

df["Month"] = df["Order_Date"].dt.to_period("M").astype(str)

monthly_sales = df.groupby("Month").agg(
    Total_Sales=("Total_Amount_INR", "sum"),
    Total_Orders=("Order_ID", "nunique")
)

print(monthly_sales.round(2))


# 13. Payment Method Analysis
print("\n" + "=" * 60)
print("12. PAYMENT METHOD ANALYSIS")
print("=" * 60)

payment_analysis = df.groupby("Payment_Method").agg(
    Total_Orders=("Order_ID", "nunique"),
```

```
    Total_Sales=("Total_Amount_INR", "sum")
).sort_values(by="Total_Sales", ascending=False)

print(payment_analysis.round(2))


# 14. City-Wise Sales Analysis
print("\n" + "=" * 60)
print("13. CITY-WISE SALES ANALYSIS")
print("=" * 60)

city_analysis = df.groupby("City").agg(
    Total_Orders=("Order_ID", "nunique"),
    Total_Sales=("Total_Amount_INR", "sum")
).sort_values(by="Total_Sales", ascending=False)

print(city_analysis.round(2))


# 15. Create a Pivot Table
print("\n" + "=" * 60)
print("14. PIVOT TABLE")
print("=" * 60)

pivot_table = pd.pivot_table(
    df,
    values="Total_Amount_INR",
    index="Category",
    columns="Order_Status",
    aggfunc="sum",
    fill_value=0
)

print(pivot_table.round(2))


# 16. Find Useful Insights
print("\n" + "=" * 60)
print("15. BUSINESS INSIGHTS")
print("=" * 60)

top_category = category_analysis["Total_Sales"].idxmax()
top_city = city_analysis["Total_Sales"].idxmax()
top_payment = payment_analysis["Total_Sales"].idxmax()

top_product = (
    df.groupby("Product")["Total_Amount_INR"]
    .sum()
    .idxmax()
)

highest_month = monthly_sales["Total_Sales"].idxmax()

print("Highest Revenue Category:", top_category)
print("Highest Revenue City:", top_city)
print("Most Valuable Payment Method:", top_payment)
print("Highest Revenue Product:", top_product)
print("Highest Sales Month:", highest_month)


# 17. Data Visualization
sns.set_theme(style="whitegrid")

# Category-wise sales chart
plt.figure(figsize=(10, 6))

category_analysis["Total_Sales"].sort_values().plot(
    kind="barh"
)
```

```
plt.title("Category-Wise Sales")
plt.xlabel("Total Sales (INR)")
plt.ylabel("Category")
plt.tight_layout()
plt.savefig("category_wise_sales.png", dpi=300)
plt.show()


# Monthly sales chart
plt.figure(figsize=(12, 6))

monthly_sales["Total_Sales"].plot(
    kind="line",
    marker="o"
)

plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales (INR)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("monthly_sales_trend.png", dpi=300)
plt.show()


# Order status chart
plt.figure(figsize=(8, 5))
df["Order_Status"].value_counts().plot(
    kind="bar"
)

plt.title("Order Status Distribution")
plt.xlabel("Order Status")
plt.ylabel("Number of Orders")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig("order_status_distribution.png", dpi=300)
plt.show()


# 18. Export the Processed Dataset
print("\n" + "=" * 60)
print("16. EXPORT PROCESSED DATA")
print("=" * 60)

df.to_csv("cleaned_ecommerce_dataset.csv", index=False)

print("Cleaned dataset exported successfully.")


# 19. Verify the Exported File
verified_df = pd.read_csv("cleaned_ecommerce_dataset.csv")

print("\nExported Dataset Shape:", verified_df.shape)
print("\nExported Dataset Preview:")
print(verified_df.head())

print("\nVerification Result:")
print("Export successful:", len(verified_df) == len(df))
print(
    "Row count matches:",
    len(verified_df) == len(df)
)

print("\nProject Completed Successfully.")
```

Output:

```
============================================================
1. ORIGINAL DATASET
============================================================
   Order_ID  Order_Date Customer_ID              Product        Category  \
0  ORD00001  2025-06-10    CUST0051  Bluetooth Headphones     Electronics
1  ORD00002  2025-09-19    CUST0113             Yoga Mat          Sports
2  ORD00003  2025-08-24    CUST0113           USB-C Cable     Electronics
3  ORD00004  2025-11-10    CUST0190             Desk Lamp  Home & Kitchen
4  ORD00005  2025-11-22    CUST0091  Bluetooth Headphones     Electronics

   Quantity  Unit_Price_INR  Discount_Percent  Discount_Amount_INR  \
0         3          1499.0              10.0               449.70
1         3           799.0              15.0               359.55
2         4           299.0              15.0               179.40
3         4           999.0               0.0                 0.00
4         4          1499.0               5.0               299.80

   Total_Amount_INR    Payment_Method Order_Status       City          State  \
0           4047.30               UPI    Delivered     Jaipur      Rajasthan
1           2037.45       Debit Card    Delivered   Bareilly  Uttar Pradesh
2           1016.60      Credit Card    Delivered     Jaipur      Rajasthan
3           3996.00  Cash on Delivery    Delivered  Ghaziabad  Uttar Pradesh
4           5696.20       Debit Card    Delivered     Kanpur  Uttar Pradesh

  Sales_Channel Delivery_Date
0    Mobile App    2025-06-17
1    Mobile App    2025-09-25
2       Website    2025-08-30
3    Mobile App    2025-11-12
4    Mobile App    2025-11-28

Dataset Shape: (1000, 16)

Column Names:
['Order_ID', 'Order_Date', 'Customer_ID', 'Product', 'Category', 'Quantity', 'Unit_Price_INR',
  'Discount_Percent', 'Discount_Amount_INR', 'Total_Amount_INR', 'Payment_Method', 'Order_Status', 'City',
  'State', 'Sales_Channel', 'Delivery_Date']

============================================================
2. DATA INSPECTION
============================================================
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 16 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Order_ID             1000 non-null   object
 1   Order_Date           1000 non-null   object
 2   Customer_ID          1000 non-null   object
 3   Product              1000 non-null   object
 4   Category             1000 non-null   object
 5   Quantity             1000 non-null   int64
 6   Unit_Price_INR       1000 non-null   float64
 7   Discount_Percent     1000 non-null   float64
 8   Discount_Amount_INR  1000 non-null   float64
 9   Total_Amount_INR     1000 non-null   float64
 10  Payment_Method       1000 non-null   object
 11  Order_Status         1000 non-null   object
 12  City                 1000 non-null   object
 13  State                1000 non-null   object
 14  Sales_Channel        1000 non-null   object
 15  Delivery_Date        831 non-null    object
dtypes: float64(4), int64(1), object(11)
memory usage: 125.1+ KB

Statistical Summary:
          Quantity  Unit_Price_INR  Discount_Percent  Discount_Amount_INR  \
count  1000.000000     1000.000000       1000.000000           1000.00000
```

```
mean      2.016000      920.400000          7.860000           145.07640
std       1.069995      691.903729          7.390874           242.91041
min       1.000000       99.000000          0.000000             0.00000
25%       1.000000      299.000000          0.000000             0.00000
50%       2.000000      799.000000          5.000000            49.80000
75%       3.000000     1499.000000         15.000000           159.90000
max       4.000000     2499.000000         20.000000          1999.20000

       Total_Amount_INR
count       1000.000000
mean        1719.607600
std         1756.177747
min           79.200000
25%          498.000000
50%         1197.000000
75%         2374.687500
max         9996.000000

Data Types:
Order_ID                object
Order_Date              object
Customer_ID             object
Product                 object
Category                object
Quantity                 int64
Unit_Price_INR         float64
Discount_Percent       float64
Discount_Amount_INR    float64
Total_Amount_INR       float64
Payment_Method          object
Order_Status            object
City                    object
State                   object
Sales_Channel           object
Delivery_Date           object
dtype: object

Duplicate Rows: 0

============================================================
3. MISSING VALUES
============================================================
Order_ID                0
Order_Date              0
Customer_ID             0
Product                 0
Category                0
Quantity                0
Unit_Price_INR          0
Discount_Percent        0
Discount_Amount_INR     0
Total_Amount_INR        0
Payment_Method          0
Order_Status            0
City                    0
State                   0
Sales_Channel           0
Delivery_Date         169
dtype: int64

Total Missing Values: 169

============================================================
4. CLEANED DATASET
============================================================
   Order_ID Order_Date Customer_ID               Product        Category  \
0  ORD00001 2025-06-10    CUST0051  Bluetooth Headphones     Electronics
1  ORD00002 2025-09-19    CUST0113             Yoga Mat          Sports
2  ORD00003 2025-08-24    CUST0113          USB-C Cable     Electronics
3  ORD00004 2025-11-10    CUST0190            Desk Lamp  Home & Kitchen
```

```
4  ORD00005 2025-11-22    CUST0091  Bluetooth Headphones     Electronics

   Quantity  Unit_Price_INR  Discount_Percent  Discount_Amount_INR  \
0         3          1499.0              10.0                449.70
1         3           799.0              15.0                359.55
2         4           299.0              15.0                179.40
3         4           999.0               0.0                  0.00
4         4          1499.0               5.0                299.80

   Total_Amount_INR    Payment_Method Order_Status       City          State  \
0           4047.30               UPI    Delivered     Jaipur      Rajasthan
1           2037.45       Debit Card    Delivered   Bareilly  Uttar Pradesh
2           1016.60      Credit Card    Delivered     Jaipur      Rajasthan
3           3996.00  Cash on Delivery    Delivered  Ghaziabad  Uttar Pradesh
4           5696.20       Debit Card    Delivered     Kanpur  Uttar Pradesh

  Sales_Channel Delivery_Date
0    Mobile App    2025-06-17
1    Mobile App    2025-09-25
2       Website    2025-08-30
3    Mobile App    2025-11-12
4    Mobile App    2025-11-28

Remaining Missing Values:
Order_ID                 0
Order_Date               0
Customer_ID              0
Product                  0
Category                 0
Quantity                 0
Unit_Price_INR           0
Discount_Percent         0
Discount_Amount_INR      0
Total_Amount_INR         0
Payment_Method           0
Order_Status             0
City                     0
State                    0
Sales_Channel            0
Delivery_Date          169
dtype: int64
============================================================
5. NUMPY STATISTICAL ANALYSIS
============================================================
Total Sales (INR): 1719607.6
Average Order Value (INR): 1719.61
Median Order Value (INR): 1197.0
Minimum Order Value (INR): 79.2
Maximum Order Value (INR): 9996.0
Sales Standard Deviation: 1755.3
Total Quantity Sold: 2016

============================================================
6. BUSINESS KPIs
============================================================
Total Orders: 1000
Unique Customers: 247
Unique Products: 15
Delivered Orders: 771
Cancelled Orders: 90
Returned Orders: 60
Cancellation Rate (%): 9.0
Return Rate (%): 6.0

============================================================
7. SELECTED COLUMNS
============================================================
   Order_ID               Product        Category  Quantity  Total_Amount_INR
0  ORD00001  Bluetooth Headphones     Electronics         3           4047.30
```

```
1  ORD00002              Yoga Mat          Sports         3           2037.45
2  ORD00003           USB-C Cable     Electronics         4           1016.60
3  ORD00004             Desk Lamp  Home & Kitchen         4           3996.00
4  ORD00005  Bluetooth Headphones     Electronics         4           5696.20
5  ORD00006        Wireless Mouse     Electronics         2           1198.00
6  ORD00007        Ballpoint Pens      Stationery         1             99.00
7  ORD00008           Smart Watch     Electronics         2           4248.30
8  ORD00009          Laptop Stand     Accessories         3           2292.45
9  ORD00010         Running Shoes         Fashion         1           1869.15

============================================================
8. FILTERING DATA
============================================================
Orders Above INR 2000:
    Order_ID Order_Date Customer_ID              Product        Category  \
0   ORD00001 2025-06-10    CUST0051  Bluetooth Headphones     Electronics
1   ORD00002 2025-09-19    CUST0113              Yoga Mat          Sports
3   ORD00004 2025-11-10    CUST0190             Desk Lamp  Home & Kitchen
4   ORD00005 2025-11-22    CUST0091  Bluetooth Headphones     Electronics
7   ORD00008 2025-12-10    CUST0158           Smart Watch     Electronics
8   ORD00009 2025-06-08    CUST0073          Laptop Stand     Accessories
11  ORD00012 2025-08-27    CUST0192  Bluetooth Headphones     Electronics
13  ORD00014 2025-12-29    CUST0015           Smart Watch     Electronics
19  ORD00020 2025-04-07    CUST0113           Smart Watch     Electronics
29  ORD00030 2025-09-23    CUST0154           Smart Watch     Electronics

    Quantity  Unit_Price_INR  Discount_Percent  Discount_Amount_INR  \
0          3          1499.0              10.0               449.70
1          3           799.0              15.0               359.55
3          4           999.0               0.0                 0.00
4          4          1499.0               5.0               299.80
7          2          2499.0              15.0               749.70
8          3           899.0              15.0               404.55
11         2          1499.0              15.0               449.70
13         1          2499.0               5.0               124.95
19         3          2499.0              20.0              1499.40
29         2          2499.0              15.0               749.70

    Total_Amount_INR    Payment_Method Order_Status       City          State  \
0            4047.30               UPI    Delivered     Jaipur      Rajasthan
1            2037.45        Debit Card    Delivered   Bareilly  Uttar Pradesh
3            3996.00  Cash on Delivery    Delivered  Ghaziabad  Uttar Pradesh
4            5696.20        Debit Card    Delivered     Kanpur  Uttar Pradesh
7            4248.30  Cash on Delivery      Shipped     Jaipur      Rajasthan
8            2292.45       Credit Card    Delivered  Ghaziabad  Uttar Pradesh
11           2548.30        Debit Card    Delivered     Jaipur      Rajasthan
13           2374.05       Net Banking    Delivered  Ghaziabad  Uttar Pradesh
19           5997.60  Cash on Delivery    Delivered   Bareilly  Uttar Pradesh
29           4248.30       Net Banking    Delivered   Gurugram        Haryana

   Sales_Channel Delivery_Date
0     Mobile App    2025-06-17
1     Mobile App    2025-09-25
3     Mobile App    2025-11-12
4     Mobile App    2025-11-28
7        Website           NaT
8    Marketplace    2025-06-15
11    Mobile App    2025-08-28
13       Website    2026-01-04
19       Website    2025-04-13
29   Marketplace    2025-09-29

Electronics Orders:
    Order_ID Order_Date Customer_ID              Product     Category  \
0   ORD00001 2025-06-10    CUST0051  Bluetooth Headphones  Electronics
2   ORD00003 2025-08-24    CUST0113           USB-C Cable  Electronics
4   ORD00005 2025-11-22    CUST0091  Bluetooth Headphones  Electronics
5   ORD00006 2025-09-29    CUST0242        Wireless Mouse  Electronics
7   ORD00008 2025-12-10    CUST0158           Smart Watch  Electronics
```

```
11  ORD00012 2025-08-27    CUST0192  Bluetooth Headphones  Electronics
13  ORD00014 2025-12-29    CUST0015           Smart Watch  Electronics
18  ORD00019 2025-05-04    CUST0124  Bluetooth Headphones  Electronics
19  ORD00020 2025-04-07    CUST0113           Smart Watch  Electronics
20  ORD00021 2025-11-26    CUST0181  Bluetooth Headphones  Electronics

    Quantity  Unit_Price_INR  Discount_Percent  Discount_Amount_INR  \
0          3          1499.0              10.0               449.70
2          4           299.0              15.0               179.40
4          4          1499.0               5.0               299.80
5          2           599.0               0.0                 0.00
7          2          2499.0              15.0               749.70
11         2          1499.0              15.0               449.70
13         1          2499.0               5.0               124.95
18         1          1499.0               0.0                 0.00
19         3          2499.0              20.0              1499.40
20         1          1499.0               5.0                74.95

    Total_Amount_INR    Payment_Method Order_Status       City          State  \
0            4047.30               UPI    Delivered     Jaipur      Rajasthan
2            1016.60       Credit Card    Delivered     Jaipur      Rajasthan
4            5696.20        Debit Card    Delivered     Kanpur  Uttar Pradesh
5            1198.00        Debit Card    Delivered     Kanpur  Uttar Pradesh
7            4248.30  Cash on Delivery      Shipped     Jaipur      Rajasthan
11           2548.30        Debit Card    Delivered     Jaipur      Rajasthan
13           2374.05       Net Banking    Delivered  Ghaziabad  Uttar Pradesh
18           1499.00  Cash on Delivery    Cancelled   Gurugram        Haryana
19           5997.60  Cash on Delivery    Delivered   Bareilly  Uttar Pradesh
20           1424.05       Credit Card    Delivered       Agra  Uttar Pradesh

   Sales_Channel Delivery_Date
0     Mobile App    2025-06-17
2        Website    2025-08-30
4     Mobile App    2025-11-28
5        Website    2025-10-02
7        Website           NaT
11    Mobile App    2025-08-28
13       Website    2026-01-04
18    Mobile App           NaT
19       Website    2025-04-13
20   Marketplace    2025-11-30
High-Value Delivered Orders:
    Order_ID Order_Date Customer_ID              Product        Category  \
0   ORD00001 2025-06-10    CUST0051  Bluetooth Headphones     Electronics
1   ORD00002 2025-09-19    CUST0113             Yoga Mat          Sports
3   ORD00004 2025-11-10    CUST0190            Desk Lamp  Home & Kitchen
4   ORD00005 2025-11-22    CUST0091  Bluetooth Headphones     Electronics
8   ORD00009 2025-06-08    CUST0073         Laptop Stand     Accessories
11  ORD00012 2025-08-27    CUST0192  Bluetooth Headphones     Electronics
13  ORD00014 2025-12-29    CUST0015           Smart Watch     Electronics
19  ORD00020 2025-04-07    CUST0113           Smart Watch     Electronics
29  ORD00030 2025-09-23    CUST0154           Smart Watch     Electronics
30  ORD00031 2025-02-21    CUST0208           Smart Watch     Electronics

    Quantity  Unit_Price_INR  Discount_Percent  Discount_Amount_INR  \
0          3          1499.0              10.0               449.70
1          3           799.0              15.0               359.55
3          4           999.0               0.0                 0.00
4          4          1499.0               5.0               299.80
8          3           899.0              15.0               404.55
11         2          1499.0              15.0               449.70
13         1          2499.0               5.0               124.95
19         3          2499.0              20.0              1499.40
29         2          2499.0              15.0               749.70
30         1          2499.0               0.0                 0.00

    Total_Amount_INR    Payment_Method Order_Status       City          State  \
0            4047.30               UPI    Delivered     Jaipur      Rajasthan
1            2037.45        Debit Card    Delivered   Bareilly  Uttar Pradesh
```

```
3            3996.00  Cash on Delivery    Delivered  Ghaziabad  Uttar Pradesh
4            5696.20        Debit Card    Delivered     Kanpur  Uttar Pradesh
8            2292.45       Credit Card    Delivered  Ghaziabad  Uttar Pradesh
11           2548.30        Debit Card    Delivered     Jaipur      Rajasthan
13           2374.05       Net Banking    Delivered  Ghaziabad  Uttar Pradesh
19           5997.60  Cash on Delivery    Delivered   Bareilly  Uttar Pradesh
29           4248.30       Net Banking    Delivered   Gurugram        Haryana
30           2499.00        Debit Card    Delivered    Lucknow  Uttar Pradesh

   Sales_Channel Delivery_Date
0     Mobile App    2025-06-17
1     Mobile App    2025-09-25
3     Mobile App    2025-11-12
4     Mobile App    2025-11-28
8    Marketplace    2025-06-15
11    Mobile App    2025-08-28
13       Website    2026-01-04
19       Website    2025-04-13
29   Marketplace    2025-09-29
30   Marketplace    2025-02-24

============================================================
9. SORTING DATA
============================================================

Top 10 Orders by Value:
     Order_ID        Product  Total_Amount_INR
903  ORD00904    Smart Watch            9996.0
486  ORD00487    Smart Watch            9996.0
531  ORD00532    Smart Watch            9996.0
729  ORD00730    Smart Watch            9996.0
790  ORD00791    Smart Watch            9996.0
325  ORD00326    Smart Watch            9996.0
184  ORD00185    Smart Watch            8996.4
78   ORD00079  Running Shoes            8796.0
781  ORD00782  Running Shoes            8796.0
316  ORD00317  Running Shoes            8796.0

Lowest 10 Orders by Value:
     Order_ID         Product  Total_Amount_INR
425  ORD00426  Ballpoint Pens             79.20
927  ORD00928  Ballpoint Pens             79.20
249  ORD00250  Ballpoint Pens             79.20
292  ORD00293  Ballpoint Pens             79.20
571  ORD00572  Ballpoint Pens             79.20
467  ORD00468  Ballpoint Pens             79.20
775  ORD00776  Ballpoint Pens             79.20
860  ORD00861  Ballpoint Pens             79.20
986  ORD00987  Ballpoint Pens             84.15
776  ORD00777  Ballpoint Pens             84.15

============================================================
10. CATEGORY-WISE ANALYSIS
============================================================
                Total_Orders  Total_Quantity  Total_Sales  Average_Order_Value
Category
Electronics              294             602    659985.05              2244.85
Fashion                  169             340    471485.40              2789.85
Accessories              128             252    249843.65              1951.90
Home & Kitchen           204             414    208938.60              1024.21
Sports                    70             127     93522.95              1336.04
Stationery               135             281     35831.95               265.42

============================================================
11. MONTHLY SALES ANALYSIS
============================================================
         Total_Sales  Total_Orders
Month
2025-01    119452.95            70
2025-02    113118.50            75
```

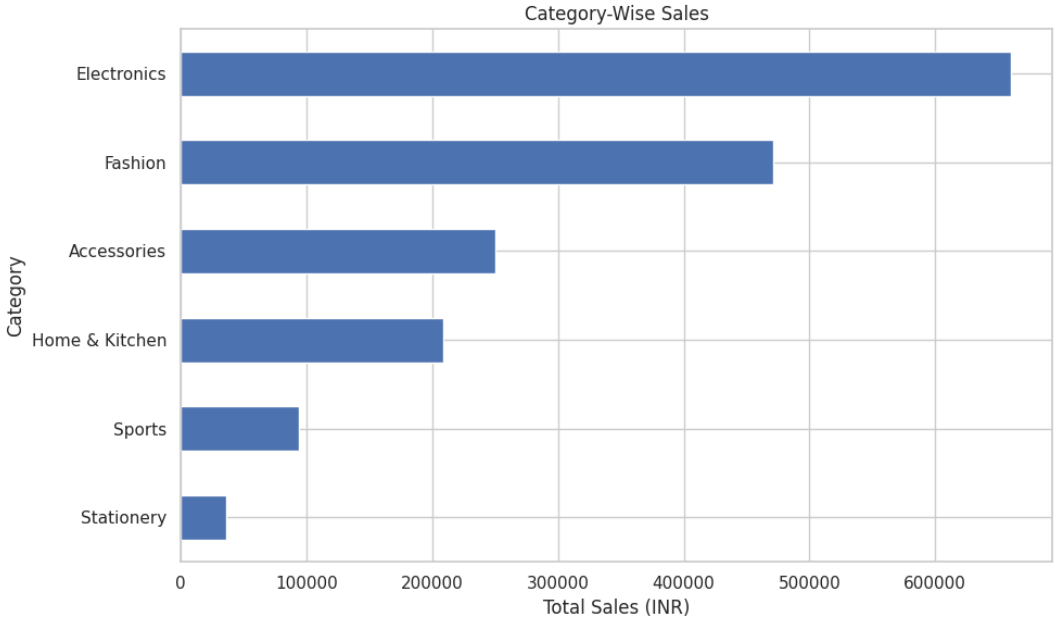
```
2025-03    152936.15            93
2025-04    159037.95            91
2025-05    161777.25            91
2025-06    151431.55            92
2025-07    143982.90            86
2025-08    168480.60            88
2025-09    121415.50            79
2025-10    125923.30            77
2025-11    148219.70            70
2025-12    153831.25            88

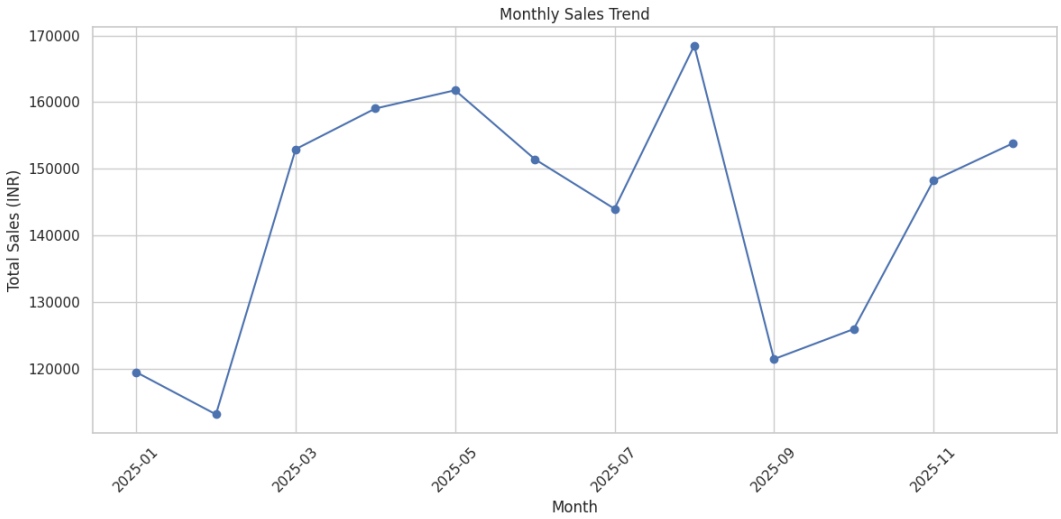
============================================================
12. PAYMENT METHOD ANALYSIS
============================================================
                 Total_Orders  Total_Sales
Payment_Method
Credit Card               221    389188.15
Net Banking               186    355678.25
Cash on Delivery          223    346693.35
Debit Card                191    338770.30
UPI                       179    289277.55

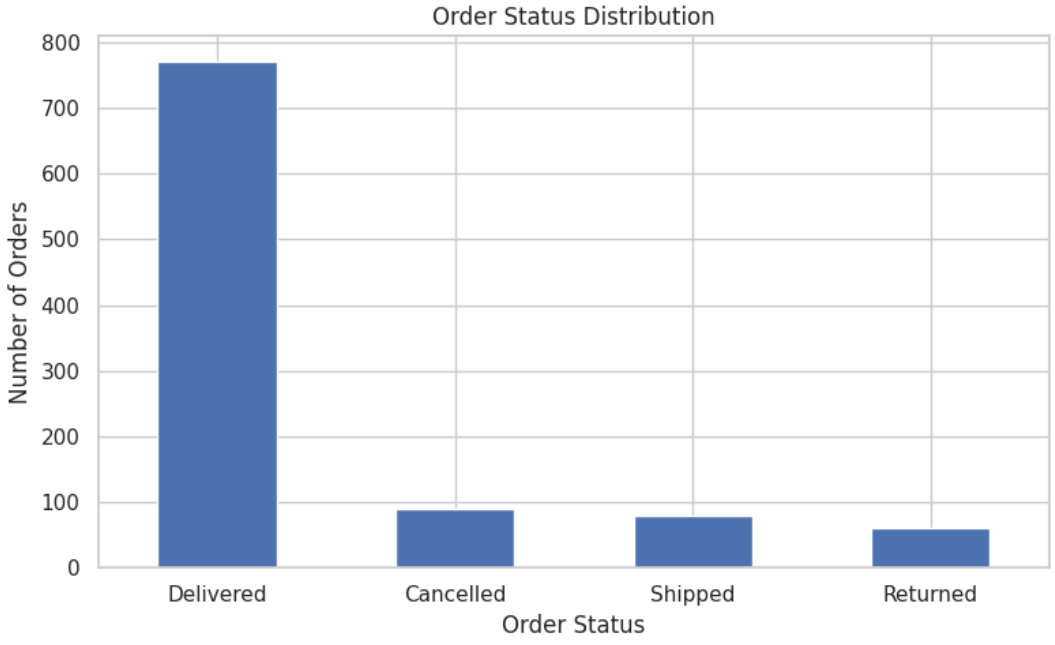
============================================================
13. CITY-WISE SALES ANALYSIS
============================================================
           Total_Orders  Total_Sales
City
Jaipur              107    209269.15
Bareilly             92    188447.65
Delhi               108    187622.55
Kanpur               98    174952.90
Noida               102    174742.65
Ghaziabad           105    174337.10
Lucknow             107    168349.10
Gurugram             88    149328.90
Agra                 94    148554.65
Mathura              99    144002.95

============================================================
14. PIVOT TABLE
============================================================
Order_Status    Cancelled  Delivered  Returned   Shipped
Category
Accessories       8272.80  209685.00  15062.05  16823.80
Electronics      81652.95  478820.30  40333.05  59178.75
Fashion          34126.80  344459.55  42684.75  50214.30
Home & Kitchen   24222.90  159122.70  13575.60  12017.40
Sports            5992.50   74386.90   3715.35   9428.20
Stationery        3566.90   27836.60   3418.65   1009.80

============================================================
15. BUSINESS INSIGHTS
============================================================
Highest Revenue Category: Electronics
Highest Revenue City: Jaipur
Most Valuable Payment Method: Credit Card
Highest Revenue Product: Smart Watch
Highest Sales Month: 2025-08
```


Category-Wise Sales
Electronics
Fashion
Accessories
Home & Kitchen
Sports
Stationery
Category
0
100000
200000
300000
400000
500000
600000
Total Sales (INR)


Monthly Sales Trend
170000
160000
150000
140000
130000
120000
Total Sales (INR)
2025-01
2025-03
2025-05
2025-07
2025-09
2025-11
Month


Order Status Distribution
800
700
600
500
400
300
200
100
0
Number of Orders
Delivered
Cancelled
Shipped
Returned
Order Status

```
============================================================
16. EXPORT PROCESSED DATA
============================================================
Cleaned dataset exported successfully.

Exported Dataset Shape: (1000, 17)

Exported Dataset Preview:
   Order_ID  Order_Date Customer_ID              Product        Category  \
0  ORD00001  2025-06-10    CUST0051  Bluetooth Headphones     Electronics
1  ORD00002  2025-09-19    CUST0113             Yoga Mat          Sports
2  ORD00003  2025-08-24    CUST0113          USB-C Cable     Electronics
3  ORD00004  2025-11-10    CUST0190            Desk Lamp  Home & Kitchen
4  ORD00005  2025-11-22    CUST0091  Bluetooth Headphones     Electronics

   Quantity  Unit_Price_INR  Discount_Percent  Discount_Amount_INR  \
0         3          1499.0              10.0               449.70
1         3           799.0              15.0               359.55
2         4           299.0              15.0               179.40
3         4           999.0               0.0                 0.00
4         4          1499.0               5.0               299.80

   Total_Amount_INR    Payment_Method Order_Status       City          State  \
0           4047.30               UPI    Delivered     Jaipur      Rajasthan
1           2037.45       Debit Card    Delivered   Bareilly  Uttar Pradesh
2           1016.60      Credit Card    Delivered     Jaipur      Rajasthan
3           3996.00  Cash on Delivery    Delivered  Ghaziabad  Uttar Pradesh
4           5696.20       Debit Card    Delivered     Kanpur  Uttar Pradesh

  Sales_Channel Delivery_Date    Month
0    Mobile App    2025-06-17  2025-06
1    Mobile App    2025-09-25  2025-09
2       Website    2025-08-30  2025-08
3    Mobile App    2025-11-12  2025-11
4    Mobile App    2025-11-28  2025-11

Verification Result:
Export successful: True
Row count matches: True

Project Completed Successfully.
```

Explanation

First, I imported NumPy, Pandas, Matplotlib, and Seaborn and loaded ecommerce.csv, which has 1000 orders and 16 columns. Then, I inspected the data using head(), shape, info(), describe(), dtypes, and checked duplicate rows (there were none).

Next, I checked missing values. Only Delivery_Date had missing values (169 rows). I did not fill these because a blank delivery date is meaningful: the order was not delivered yet or was cancelled. I converted Order_Date and Delivery_Date to datetime, and wrote code to fill any missing text values with 'Unknown' and missing numeric values with the median, although none were needed in this dataset.

After that, I used NumPy functions to calculate total sales, average, median, minimum, maximum, and standard deviation of order value, and calculated KPIs such as number of customers, delivered, cancelled, and returned orders with their rates.

Then, I selected specific columns, filtered orders (above INR 2000, Electronics, and high-value delivered orders), and sorted orders to find the top 10 and lowest 10 by value. I used groupby() to analyse sales by category, month, payment method, and city, and created a pivot table of category against order status.

Finally, I created three charts (category-wise sales, monthly sales trend, and order status distribution), exported the cleaned dataset to cleaned_ecommerce_dataset.csv, and read it back to verify that the row count matched.

Key Findings / Insights (Task 10)

- **Total sales were INR 17,19,607.60 from 1000 orders placed by 247 customers. The average order value was INR 1,719.61 and the median was INR 1,197, so a few high-value orders pull the average above the median.**
- **Electronics is the top category with INR 6,59,985 in sales (about 38% of total sales). Fashion is second with INR 4,71,485 (about 27%), and Accessories is third. These three categories together give about 80% of the revenue.**
- **Fashion has the highest average order value (INR 2,789.85), while Stationery has the lowest (INR 265.42) and the lowest total sales (INR 35,832) even though it has 135 orders.**
- **Smart Watch is the highest revenue product. The maximum order value is INR 9,996, which is a Smart Watch order of 4 units.**
- **Jaipur is the highest revenue city (INR 2,09,269), followed by Bareilly and Delhi. Delhi has the most orders (108), so Jaipur wins because of a higher value per order.**
- **Credit Card gives the highest sales (INR 3,89,188), while Cash on Delivery has the highest number of orders (223). UPI has the lowest sales value (INR 2,89,278).**
- **August 2025 is the best month (INR 1,68,481) and February 2025 is the weakest month (INR 1,13,119). Sales also dip in September and October compared with the months before.**
- **Out of 1000 orders, 771 were delivered, 90 were cancelled (9%), 60 were returned (6%), and the remaining 79 were still shipped. The 169 missing Delivery_Date values match the 90 cancelled and 79 shipped orders.**
- **Electronics lost the most revenue to cancellations (INR 81,653, about 12% of its sales). Fashion has the highest return rate by value (INR 42,685, about 9% of its sales).**

Datasets Used

- **student.csv: 10 students with Student_ID, Name, Age, Marks, and Attendance (Tasks 5 and 6).**
- **ecommerce.csv: 1000 orders with 16 columns (Task 10). The cleaned file is saved as cleaned_ecommerce_dataset.csv.**
- **processed_students.csv: exported file from Task 9.**

Google Drive Link

(Paste your Google Drive link here. Make sure it has the required access permission.)